claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_085a5118-2cb\agent-aadea9cd961e6cab0.jsonl`
- SHA-256: `785fe1193876cd4c3008a4efcd26bba61afc47d6342d61691a8ee1eef633a1e6`
- Source modified: `2026-06-20T16:57:49+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_085a5118-2cb`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`

Transcript

1. user (2026-06-20T16:56:24.752Z)

Map EVERY per-video artifact write site in the LOCAL pipeline/segmenter path. Read backend/pipeline/segmenters/*.py (especially trajectory_hybrid.py and yolo_hybrid.py) and any base.py. Find every place a directory is created or a file is written for a video: trajectory CSVs, candidate CSVs, handoff/clip chunks, frame caches, intermediate dirs. Note the literal "output"/"output/<family>"/"output/chunks_*_handoff" hardcodes (trajectory_hybrid.py ~237 and ~313). Record exact file:line, the path pattern, whether it is hardcoded, and what it is keyed by (video_id/family/none).

2. assistant (2026-06-20T16:56:26.658Z)

I'll help you map every per-video artifact write site in the local pipeline/segmenter path. Let me start by exploring the structure and reading the relevant files.

3. user (2026-06-20T16:56:27.129Z)

backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters__init__.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters__init__.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\base.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\gemini.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\motion.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters__init__.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\base.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\motion.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\gemini.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\base.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\gemini.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\motion.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters__init__.py

.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\base.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\gemini.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\motion.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters__init__.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\hardchop\backend\pipeline\segmenters__init__.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\base.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\gemini.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\motion.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\hardchop\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\base.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\gemini.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\motion.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters__init__.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters__init__.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\base.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\gemini.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\motion.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\pr-stitch\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\boxhill17\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters__init__.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\base.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\gemini.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\motion.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\gap-doc\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\resweep\backend\pipeline\segmenters__init__.py
.claude\worktrees\resweep\backend\pipeline\segmenters\base.py
.claude\worktrees\resweep\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\resweep\backend\pipeline\segmenters\gemini.py
.claude\worktrees\resweep\backend\pipeline\segmenters\motion.py
.claude\worktrees\resweep\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\resweep\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\resweep\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\resweep\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\base.py
.claude\worktrees\recdoc\backend\pipeline\segmenters__init__.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\gemini.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\motion.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\recdoc\backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\motion.py

```
backend\pipeline\segmenters\_keyhint.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
```

4. user (2026-06-20T16:56:27.973Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/audio/hit_detector.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/audio/__init__.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/audio_features.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/base.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/fusion_features.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/person_detector.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/rally_gate.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/tracknet_runner.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_infer.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/__init__.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/personalization/serving.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/personalization/__init__.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/base.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/fusion_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/gemini.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/motion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/tracknet_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/backend/pipeline/segmenters/trajectory_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/wasb_hybrid.py
```

5. assistant (2026-06-20T16:56:30.394Z)

Now let me read the main segmenter files in the primary repo (not worktrees), focusing on trajectory_hybrid.py, yolo_hybrid.py, and base.py:

6. user (2026-06-20T16:56:30.782Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
```

```
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47          #: config section under `indexing` (velocity_threshold lives there), the
48          #: output subdir for trajectories, and the metadata detector-tag prefix.
49          family: str = ""
50          #: human-readable label used in log lines.
51          display_label: str = ""
52
53          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54              """Return (runner, detector-specific detection_params extras) for the default
(WSL) path."""
55              raise NotImplementedError
56
57          def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
58              """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
with a
59              native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
60              raise NotImplementedError(
61                  f"detector_impl='native' is not supported for the "
62                  f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63                  f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65          def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
```

```
Dict[str, Any]]:
66              """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68              ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69              ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70              is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71              in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72              ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73              (the WSL runner) so the default production path is unchanged
74              (``docs/DECOUPLED_COMPUTE/``).
75              """
76              impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77              if impl == "stub":
78                  from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79                  logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                              self.display_label or "trajectory")
81                  return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82              if impl in ("native", "native_wasb", "wasb_native"):
83                  logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                              self.display_label or "trajectory")
85                  return self._build_native_runner(idx_cfg)
86              return self._build_runner(idx_cfg)
87
88          @staticmethod
89          def _probe_fps_width(video_path: str) -> tuple:
90              """Return (fps, frame_width) for a video, with sensible fallbacks."""
91              cap = cv2.VideoCapture(video_path)
92              fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93              width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94              cap.release()
95              return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97          @staticmethod
98          def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99              """Provider-reported exchange count, else a duration-based estimate.
100
101             One canonical implementation ? the twins had drifted (wasb relied on
102             ``int(None)`` raising into the fallback; same result, by accident).
103             """
104             shot_count = segment.get("shuttle_exchange_count")
105             if shot_count is None:
106                 return max(3, int(duration / 0.8))
107             try:
108                 return int(shot_count)
109             except (ValueError, TypeError):
110                 return max(3, int(duration / 0.8))
111
112         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
113         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                     frame_width: float, video_path: str,
115                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
```

```
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
            # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
            # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
            # measure it against the Gemini path, or to avoid the cloud entirely). With
            # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
            skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            if skip_ai:
                model_name = "local-noai"
                logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
                            "be emitted as rallies; no %s handoff, no API key needed.",
                            label, provider_name)
            else:
                model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    from backend.pipeline.segmenters._keyhint import api_key_hint
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        + api_key_hint(api_key_env_var)
```

```
                    )
                    return []
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
            # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
            # it delegates to the subclass _build_runner (the real WSL/native runner).
            try:
                runner, runner_params = self._resolve_runner(idx_cfg)
            except NotImplementedError as e:
                # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
                # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
                logger.error("%s: %s", label, e)
                return []

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s detector environment not ready: %s", label, msg)
                return []
            logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
```

```
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
```

```
gated.
293             handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
294
295             ai_segments: List[dict] = []
296             if skip_ai:
297                 # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
298                 # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
299                 # falls back to the duration-based estimate (no AI exchange count).
300                 ai_segments = [
301                     {
302                         "start_time": all_candidate_windows[wi]["start"],
303                         "end_time": all_candidate_windows[wi]["end"],
304                         "shuttle_exchange_count": None,
305                         "shot_count_confidence": "LocalNoAI",
306                         "_candidate_id": candidate_ids[wi],
307                     }
308                     for wi in handoff_indices
309                 ]
310                 logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
311                             "(no AI handoff).", len(ai_segments))
312             else:
313                 handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
314                 os.makedirs(handoff_dir, exist_ok=True)
315                 logger.info("Phase 3: %s validation on %d candidate windows...",
316                             provider_name, len(handoff_indices))
317
318                 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
319                     w_start = max(0, window["start"] - handoff_padding)
320                     w_end = min(video_duration, window["end"] + handoff_padding)
321                     w_dur = w_end - w_start
322                     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
323                     if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
324                         return []
325                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
326                     segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
327                                                          label=f"Handoff {wi+1}")
328                     valid = []
329                     for seg in segments:
330                         try:
331                             seg["start_time"] = float(seg["start_time"]) + w_start
332                             seg["end_time"] = float(seg["end_time"]) + w_start
333                             seg["_candidate_id"] = candidate_ids[wi]
334                             valid.append(seg)
335                         except (ValueError, TypeError, KeyError) as e:
336                             logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
337                     try:
338                         os.remove(w_path)
339                     except OSError:
340                         pass
341                     return valid
342
343                 with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
344                     futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
345                                for wi in handoff_indices]
346                     for future in futures:
347                         ai_segments.extend(future.result())
```

```
348                 try:
349                     os.rmdir(handoff_dir)
350                 except OSError:
351                     pass
352                 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
353 
354             # --- Phase 4: DB population + candidate linking ---
355             segments_data = []
356             for segment in ai_segments:
357                 try:
358                     start = float(segment["start_time"])
359                     end = float(segment["end_time"])
360                     dur = end - start
361                     metadata = {
362                         "shot_count": self._shot_count_from(segment, dur),
363                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
364                         "detector": f"{self.family}-hybrid-{model_name}",
365                     }
366                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
367                     if segment_id:
368                         candidate_id = segment.get("_candidate_id")
369                         if candidate_id is not None:
370                             self.db.link_candidate_to_segment(candidate_id, segment_id)
371                         segments_data.append({
372                             "id": segment_id, "start_time": start, "end_time": end,
373                             "duration": dur, "metadata": metadata,
374                         })
375                 except Exception as e:
376                     logger.warning("Skipping invalid segment struct: %s", e)
377 
378             return segments_data
379 
```

7. user (2026-06-20T16:56:31.068Z)

```
1       """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2       (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3 
4       Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5       (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6       can be consumed by the future fusion segmenter; this segmenter just orchestrates
7       chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8 
9       Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10      service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11      detection uses torchvision's permissively-licensed (BSD) detectors and the
12      association/tracking that YOLO.track() used to bundle in is provided by
13      supervision's MIT-licensed ByteTrack.
14 
15      The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16      DB `detector` tags, even though no YOLO is involved anymore.
17      """
18      import os
19      import time
20      import logging
21      import concurrent.futures
22      from typing import List, Dict, Any, Tuple
23 
24      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25      from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26      from backend.utils.video import get_video_duration, extract_video_chunk
27      from backend.sports import sport_registry
```

```
    from backend.providers import provider_registry

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class HybridYoloSegmenter(BasePlaySegmenter):
        def __init__(self, db, config):
            super().__init__(db, config)
            # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
producer
            # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
            self.person = PersonDetector(config)

        @property
        def name(self) -> str:
            return "yolo_hybrid"

        @property
        def description(self) -> str:
            return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
                    "candidate filtering, then an AI provider for high-precision semantic
boundaries.")

        def _resolve_provider(self, video_path: str):
            """Phase 0: resolve sport profile + AI provider (with API key).

            Returns the configured provider instance, or ``None`` to signal that
            ``process_video`` should early-return ``[]`` (bad sport, missing API key,
            or unknown provider). Behaviour is identical to the inlined version.
            """
            # Get sport profile config
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return None

            # Get AI provider and model config
            idx_cfg = self.config.get("indexing", {})
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

            # Get appropriate API key based on the provider
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                from backend.pipeline.segmenters._keyhint import api_key_hint
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} segmenter, set your API key. "
                    + api_key_hint(api_key_env_var)
                )
                return None

```

```
            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return None

            logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
            return provider

        def _collect_candidate_windows(
            self, video_id: str, video_path: str, video_duration: float, idx_cfg: Dict[str,
Any],
            chunk_duration: float, chunk_overlap: float, velocity_thresh: float, merge_gap:
float,
            frame_skip: int, chunks_dir: str,
        ) -> List[Dict[str, Any]]:
            """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
            sequential), then dedup-merge overlapping windows across chunks. Verbatim
move."""
            step = chunk_duration - chunk_overlap
            chunk_starts = []
            t = 0.0
            while t < video_duration:
                chunk_starts.append(t)
                t += step

            num_chunks = len(chunk_starts)
            logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")

            all_candidate_windows = []

            t_start = time.time()
            prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)

            if num_chunks > 1 and prefetch_workers > 0:
                # --- PIPELINED PROCESSING ---
                # GPU inference must stay sequential (single GPU), but ffmpeg chunk
                # extraction is pure I/O. We pre-extract upcoming chunks in background
                # threads so extraction of chunk N+1 overlaps with inference on chunk N.
                logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")

                extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)

                def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
                    chunk_end = min(cs + chunk_duration, video_duration)
                    actual_dur = chunk_end - cs
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
                    success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
                    return (ci, cs, chunk_path, success)

                # Submit all extractions upfront ? they'll queue and run as workers free up
                extract_futures = [
                    extract_executor.submit(_extract_chunk, ci, cs)
                    for ci, cs in enumerate(chunk_starts)
                ]

                # Process results in order: wait for extraction, then run inference on GPU
                for future in extract_futures:
                    ci, chunk_start, chunk_path, success = future.result()
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
```

```
                    logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")

                    if not success:
                        logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path, chunk_start, velocity_thresh, merge_gap,
                        frame_skip=frame_skip, chunk_index=ci
                    )
                    logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                extract_executor.shutdown(wait=False)
            else:
                # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
                for ci, chunk_start in enumerate(chunk_starts):
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_dur = chunk_end - chunk_start
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
                    success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
                    if not success:
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path, chunk_start, velocity_thresh, merge_gap,
                        frame_skip=frame_skip, chunk_index=ci
                    )
                    logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

            # Deduplicate overlapping candidate windows across chunks (chunks overlap by
            # chunk_overlap, so the same rally can surface in two adjacent chunks).
            def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
                fa, fb = a["active_frame_count"], b["active_frame_count"]
                total = fa + fb or 1
                return {
                    "start": a["start"],
                    "end": max(a["end"], b["end"]),
                    "active_frame_count": fa + fb,
                    "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
                    # Frame-weighted mean so longer sub-windows dominate proportionally.
                    "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
                    "track_id_count": max(a["track_id_count"], b["track_id_count"]),
                    "chunk_index": a["chunk_index"],
                }

```

```
205             if all_candidate_windows:
206                 all_candidate_windows.sort(key=lambda w: w["start"])
207                 merged_windows = [all_candidate_windows[0]]
208                 for current in all_candidate_windows[1:]:
209                     last = merged_windows[-1]
210                     if current["start"] <= last["end"]:  # Overlap
211                         merged_windows[-1] = _merge_stats(last, current)
212                     else:
213                         merged_windows.append(current)
214                 all_candidate_windows = merged_windows
215 
216             logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
217             logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
218 
219             return all_candidate_windows
220 
221         def _persist_candidates(
222             self, video_id: str, all_candidate_windows: List[Dict[str, Any]],
223             detection_params: Dict[str, Any], log_candidates: bool, store_candidates: bool,
224         ) -> List[Any]:
225             """Phase 2b: per-rally console log + persist raw candidates for the fine-tuning
226             dataset. Returns window-index -> candidate_id (None where not stored). Verbatim
move."""
227             # Per-rally console log (final, full-video timestamps) for video cross-
verification.
228             if log_candidates:
229                 for w in all_candidate_windows:
230                     logger.info(
231                         f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
232                         f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
233                         f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
234                         f"tracks={w['track_id_count']}"
235                     )
236 
237             # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
238             # so confirmed AI segments can be linked back in Phase 4.
239             candidate_ids = [None] * len(all_candidate_windows)
240             if store_candidates:
241                 for i, w in enumerate(all_candidate_windows):
242                     stats = {
243                         "peak_velocity": w["peak_velocity"],
244                         "mean_velocity": w["mean_velocity"],
245                         "active_frame_count": w["active_frame_count"],
246                         "track_id_count": w["track_id_count"],
247                     }
248                     # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
249                     confidence = min(1.0, w["peak_velocity"])
250                     candidate_ids[i] = self.db.add_candidate_segment(
251                         video_id, detector="yolo_hybrid",
252                         start_time=w["start"], end_time=w["end"],
253                         chunk_index=w["chunk_index"], confidence=confidence,
254                         stats=stats, detection_params=detection_params,
255                     )
256 
257             return candidate_ids
258 
259         def _run_ai_handoff(
260             self, video_id: str, video_path: str, video_duration: float,
261             all_candidate_windows: List[Dict[str, Any]], candidate_ids: List[Any],
262             provider, sport_profile, provider_name: str, chunk_duration: float,
263             handoff_padding: float, ai_max_workers: int,
264         ) -> List[dict]:
```

```
            """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
            the provider for semantic boundaries, in parallel. Verbatim move."""
            gemini_segments = []
            gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(gemini_dir, exist_ok=True)

            logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start

                w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
                # Re-encode short handoff clips to avoid keyframe-boundary drift
                success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
                if not success:
                    return []

                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")

                # Coordinate mapping ? shift local timestamps back to full-video time
                valid_segments = []
                for segment in segments:
                    try:
                        segment["start_time"] = float(segment["start_time"]) + w_start
                        segment["end_time"] = float(segment["end_time"]) + w_start
                        # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
                        segment["_candidate_id"] = candidate_ids[wi]
                        valid_segments.append(segment)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")

                try:
                    os.remove(w_path)
                except OSError:
                    pass

                return valid_segments

            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
                # Iterate in submission order to preserve temporal ordering
                for future in futures:
                    gemini_segments.extend(future.result())

            try:
                os.rmdir(gemini_dir)
            except OSError:
                pass

            logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
            return gemini_segments

        def _populate_db(self, video_id: str, gemini_segments: List[dict], model_name: str)
-> List[Dict[str, Any]]:
```

```
            """Phase 4: DB Population ? write confirmed AI segments, link them back to their
            source candidate, and build the returned segments_data. Verbatim move."""
            segments_data = []
            for segment in gemini_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start

                    shot_count = segment.get("shuttle_exchange_count")
                    if shot_count is None:
                        shot_count = max(3, int(dur / 0.8))
                    else:
                        try:
                            shot_count = int(shot_count)
                        except (ValueError, TypeError):
                            shot_count = max(3, int(dur / 0.8))

                    metadata = {
                        "shot_count": shot_count,
                        "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
                        "detector": f"yolo-hybrid-{model_name}"
                    }

                    segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
                    if segment_id:
                        # Link back to the candidate that produced this segment. A candidate
                        # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
                        # the last one ? the link table records "this candidate was
confirmed".
                        candidate_id = segment.get("_candidate_id")
                        if candidate_id is not None:
                            self.db.link_candidate_to_segment(candidate_id, segment_id)
                        segments_data.append({
                            "id": segment_id,
                            "start_time": start,
                            "end_time": end,
                            "duration": dur,
                            "metadata": metadata
                        })
                except Exception as e:
                    logger.warning(f"Skipping invalid segment struct: {e}")

            return segments_data

        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
            self.person.lazy_load_model()

            # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
            provider = self._resolve_provider(video_path)
            if provider is None:
                return []

            # Re-read the config values needed downstream (pure deterministic reads).
            idx_cfg = self.config.get("indexing", {})
            sport_profile = sport_registry.get(self.config.get("sport", "badminton"))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
```

```
378
379             video_duration = get_video_duration(video_path)
380             if video_duration <= 0:
381                 logger.error("Invalid video duration.")
382                 return []
383
384             chunk_duration = idx_cfg.get("chunk_duration", 300)
385             chunk_overlap = idx_cfg.get("chunk_overlap", 30)
386             velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15)  # Fraction of
frame width per second
387             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
388             frame_skip = idx_cfg.get("yolo_frame_skip", 3)
389             tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
390             handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
391             ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
392             log_candidates = idx_cfg.get("log_yolo_candidates", True)
393             store_candidates = idx_cfg.get("store_yolo_candidates", True)
394
395             # Snapshot of the params that produced these detections ? stored alongside each
396             # candidate so a fine-tuning run can reproduce / correlate against them.
397             detection_params = {
398                 "velocity_thresh": velocity_thresh,
399                 "merge_gap": merge_gap,
400                 "frame_skip": frame_skip,
401                 "tracker": tracker_config,
402                 "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
403                 "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
404                 "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
405             }
406
407             chunks_dir = os.path.join("output", "chunks_yolo")
408             os.makedirs(chunks_dir, exist_ok=True)
409
410             # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
411             all_candidate_windows = self._collect_candidate_windows(
412                 video_id, video_path, video_duration, idx_cfg,
413                 chunk_duration, chunk_overlap, velocity_thresh, merge_gap,
414                 frame_skip, chunks_dir,
415             )
416
417             # Phase 2b: log + persist raw candidates (window index -> candidate_id).
418             candidate_ids = self._persist_candidates(
419                 video_id, all_candidate_windows, detection_params, log_candidates,
store_candidates,
420             )
421
422             try:
423                 os.rmdir(chunks_dir)
424             except Exception:
425                 pass
426
427             # Phase 3: AI Provider Handoff
428             gemini_segments = self._run_ai_handoff(
429                 video_id, video_path, video_duration, all_candidate_windows, candidate_ids,
430                 provider, sport_profile, provider_name, chunk_duration, handoff_padding,
ai_max_workers,
431             )
432
433             # Phase 4: DB Population
434             return self._populate_db(video_id, gemini_segments, model_name)
435
436     # Register the segmenter
```

```
segmenter_registry.register(HybridYoloSegmenter)

```

8. user (2026-06-20T16:56:31.241Z)

```
      from abc import ABC, abstractmethod
      from typing import List, Dict, Any, Type, Optional
      from backend.infrastructure.database import Database
      from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)

      class BasePlaySegmenter(ABC):
          def __init__(self, db: Database, config: Dict[str, Any]):
              self.db = db
              self.config = config

          @property
          @abstractmethod
          def name(self) -> str:
              """Unique identifier for this segmenter."""
              pass

          @property
          @abstractmethod
          def description(self) -> str:
              """Human-readable explanation of this segmenter."""
              pass

          @abstractmethod
          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
              """
              Processes a video to detect play segments, populates SQLite database,
              and returns a Result.

              Use isinstance(result, Failure) (or result.is_success()) to distinguish
              "no segments found" from "something went wrong" (missing key, healthcheck
              failure, etc.). This is the start of the standardized error/result contract.
              """
              pass

      class SegmenterRegistry:
          def __init__(self):
              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
              """Registers a new segmenter class under its name property."""
              try:
                  temp_inst = segmenter_cls(None, {})
                  name = temp_inst.name
              except Exception as e:
                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
              self._segmenters[name] = segmenter_cls

          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
              return self._segmenters.get(name)

          def list_available(self) -> Dict[str, str]:
              """Returns a dict of registered segmenter names and their descriptions."""
              res = {}
              for name, cls in self._segmenters.items():
                  try:
                      temp_inst = cls(None, {})
                      desc = temp_inst.description
```

```
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

9. assistant (2026-06-20T16:56:35.797Z)

Now let me search for other write sites, particularly in the detector files that might write artifacts:

10. user (2026-06-20T16:56:36.870Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-5-import concurrent.futures
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-6-from
typing import List, Dict, Any, Tuple, Callable
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-7-from
backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
backend\pipeline\segmenters\gemini.py:8:from backend.utils.video import get_video_duration,
extract_video_chunk
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-9-from
backend.sports import sport_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-10-from backend.providers import provider_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py-11-from backend.reporting import run_signals
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-20-
otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-21-
in-flight work. A per-video subdir means each run only ever creates/cleans its own.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-22-
"""
backend\pipeline\segmenters\gemini.py:23:    return os.path.join("output", "chunks", video_id)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-24-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-25-
backend\pipeline\segmenters\gemini.py-26-def _parse_time(val) -> float:
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-116-
backend\pipeline\segmenters\gemini.py-117-        if debug_duration:
backend\pipeline\segmenters\gemini.py-118-            logger.info(f"DEBUG FEATURE: Trimming
video to first {debug_duration} seconds for fast processing...")
backend\pipeline\segmenters\gemini.py:119:            temp_trimmed_path = os.path.join("output",
f"temp_trimmed_{video_id}.mp4")
backend\pipeline\segmenters\gemini.py:120:            os.makedirs("output", exist_ok=True)
backend\pipeline\segmenters\gemini.py-121-            if os.path.exists(temp_trimmed_path):
backend\pipeline\segmenters\gemini.py-122-                try:
backend\pipeline\segmenters\gemini.py:123:                    os.remove(temp_trimmed_path)
backend\pipeline\segmenters\gemini.py-124-                except Exception:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-125-
pass
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-126-
--
backend\pipeline\segmenters\gemini.py-145-        def cleanup_temp_files():
backend\pipeline\segmenters\gemini.py-146-            if is_trimmed and temp_trimmed_path and
os.path.exists(temp_trimmed_path):
backend\pipeline\segmenters\gemini.py-147-                try:
backend\pipeline\segmenters\gemini.py:148:                    os.remove(temp_trimmed_path)
backend\pipeline\segmenters\gemini.py-149-                except Exception:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-150-
pass
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-151-
```

```
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-179-
f"({chunk_duration}s each, {chunk_overlap}s overlap)."
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-180-
)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-181-
backend\pipeline\segmenters\gemini.py:182:        os.makedirs(chunks_dir, exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-183-
backend\pipeline\segmenters\gemini.py-184-        def process_single_chunk(ci: int, chunk_start:
float) -> Tuple[str, list, dict]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-185-
chunk_end = min(chunk_start + chunk_duration, video_duration)
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-189-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-190-
# Extract chunk with ffmpeg
backend\pipeline\segmenters\gemini.py-191-            logger.info(f"Extracting {chunk_label}:
{chunk_start:.1f}s - {chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
backend\pipeline\segmenters\gemini.py:192:            success = extract_video_chunk(
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-193-
video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-194-
reencode=ai_scale_width > 0,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-195-
scale_width=ai_scale_width if ai_scale_width > 0 else None,
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-231-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-232-
# Clean up chunk file
backend\pipeline\segmenters\gemini.py-233-            try:
backend\pipeline\segmenters\gemini.py:234:                os.remove(chunk_path)
backend\pipeline\segmenters\gemini.py-235-            except Exception:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-236-
pass
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py-237-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-26-import cv2
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-27-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-28-from
backend.pipeline.segmenters.base import BasePlaySegmenter
backend\pipeline\segmenters\trajectory_hybrid.py:29:from backend.utils.video import
get_video_duration, extract_video_chunk
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-30-from backend.sports import
sport_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-31-from backend.providers import
provider_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-32-from backend.pipeline.detectors.base
import DetectorRunner
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-234-        # Trajectory detectors
process the whole video in one pass (they carry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-235-        # their own frame
buffering), so unlike yolo_hybrid we don't pre-chunk.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-236-        t_start = time.time()
backend\pipeline\segmenters\trajectory_hybrid.py:237:        out_dir = os.path.join("output",
self.family)
```

```
backend\pipeline\segmenters\trajectory_hybrid.py:238:        csv_path =
runner.run_predict(video_path, out_dir, video_id=video_id)
backend\pipeline\segmenters\trajectory_hybrid.py:239:        if not csv_path:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-240-            logger.error("%s
produced no trajectory; aborting.", label)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-241-            return []
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-242-
backend\pipeline\segmenters\trajectory_hybrid.py:243:        points =
runner.parse_trajectory_csv(csv_path)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-244-        logger.info("Parsed %d
trajectory points (%d visible).",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-245-                    len(points),
sum(1 for p in points if p.visible))
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-246-
--
backend\pipeline\segmenters\trajectory_hybrid.py-310-            logger.info("Phase 3 SKIPPED
(fully-local): emitted %d candidate windows as rallies "
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-311-                        "(no AI
handoff).", len(ai_segments))
backend\pipeline\segmenters\trajectory_hybrid.py-312-        else:
backend\pipeline\segmenters\trajectory_hybrid.py:313:            handoff_dir =
os.path.join("output", f"chunks_{provider_name}_handoff")
backend\pipeline\segmenters\trajectory_hybrid.py:314:            os.makedirs(handoff_dir,
exist_ok=True)
backend\pipeline\segmenters\trajectory_hybrid.py-315-            logger.info("Phase 3: %s
validation on %d candidate windows...",
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-316-
provider_name, len(handoff_indices))
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-317-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-320-                w_end =
min(video_duration, window["end"] + handoff_padding)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-321-                w_dur = w_end -
w_start
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-322-                w_path =
os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
backend\pipeline\segmenters\trajectory_hybrid.py:323:                if not
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-324-                    return []
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-325-                prompt =
sport_profile.get_prompt(chunk_duration=w_dur)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-326-                segments, _ =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
--
backend\pipeline\segmenters\trajectory_hybrid.py-335-                    except (ValueError,
TypeError, KeyError) as e:
backend\pipeline\segmenters\trajectory_hybrid.py-336-
logger.warning("Dropping segment with unparseable timestamps: %s ? %s", seg, e)
backend\pipeline\segmenters\trajectory_hybrid.py-337-                try:
backend\pipeline\segmenters\trajectory_hybrid.py:338:                    os.remove(w_path)
```

```
backend\pipeline\segmenters\trajectory_hybrid.py-339-                except OSError:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-340-                    pass
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py-341-                return valid
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-23-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-24-from backend.pipeline.segmenters.base
import BasePlaySegmenter, segmenter_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-25-from
backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
backend\pipeline\segmenters\yolo_hybrid.py:26:from backend.utils.video import
get_video_duration, extract_video_chunk
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-27-from backend.sports import sport_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-28-from backend.providers import
provider_registry
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-29-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-130-                chunk_end = min(cs +
chunk_duration, video_duration)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-131-                actual_dur = chunk_end -
cs
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-132-                chunk_path =
os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
backend\pipeline\segmenters\yolo_hybrid.py:133:                success =
extract_video_chunk(video_path, cs, actual_dur, chunk_path)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-134-                return (ci, cs,
chunk_path, success)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-135-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-136-            # Submit all extractions
upfront ? they'll queue and run as workers free up
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-157-
all_candidate_windows.extend(windows)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-158-
backend\pipeline\segmenters\yolo_hybrid.py-159-                try:
backend\pipeline\segmenters\yolo_hybrid.py:160:                    os.remove(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py-161-                except Exception:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-162-                    pass
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-163-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-170-                chunk_path =
os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-171-
backend\pipeline\segmenters\yolo_hybrid.py-172-                logger.info(f"Processing Chunk
{ci+1}\{num_chunks} ({chunk_start:.1f}s - {chunk_end:.1f}s)...")
backend\pipeline\segmenters\yolo_hybrid.py:173:                success =
```

```
extract_video_chunk(video_path, chunk_start, actual_dur, chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py-174-                if not success:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-175-                    continue
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-176-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-182-
all_candidate_windows.extend(windows)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-183-
backend\pipeline\segmenters\yolo_hybrid.py-184-                try:
backend\pipeline\segmenters\yolo_hybrid.py:185:                    os.remove(chunk_path)
backend\pipeline\segmenters\yolo_hybrid.py-186-                except Exception:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-187-                    pass
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-188-
--
backend\pipeline\segmenters\yolo_hybrid.py-265-        """Phase 3: AI Provider Handoff ? re-
encode each padded candidate window and ask
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-266-        the provider for semantic
boundaries, in parallel. Verbatim move."""
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-267-        gemini_segments = []
backend\pipeline\segmenters\yolo_hybrid.py:268:        gemini_dir = os.path.join("output",
f"chunks_{provider_name}_handoff")
backend\pipeline\segmenters\yolo_hybrid.py:269:        os.makedirs(gemini_dir, exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-270-
backend\pipeline\segmenters\yolo_hybrid.py-271-        logger.info(f"Starting Phase 3: AI
Provider ({provider_name}) Validation on {len(all_candidate_windows)} candidate windows...")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-272-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-277-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-278-            w_path =
os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-279-            # Re-encode short handoff
clips to avoid keyframe-boundary drift
backend\pipeline\segmenters\yolo_hybrid.py:280:            success =
extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True)
backend\pipeline\segmenters\yolo_hybrid.py-281-            if not success:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-282-                return []
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-283-
--
backend\pipeline\segmenters\yolo_hybrid.py-297-                    logger.warning(f"Dropping
segment with unparseable timestamps: {segment} ? {e}")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-298-
backend\pipeline\segmenters\yolo_hybrid.py-299-            try:
backend\pipeline\segmenters\yolo_hybrid.py:300:                os.remove(w_path)
backend\pipeline\segmenters\yolo_hybrid.py-301-            except OSError:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-302-                pass
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-303-
--
```

```
backend\pipeline\segmenters\yolo_hybrid.py-404-            "min_action_window_duration":
idx_cfg.get("min_action_window_duration", 2.0),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-405-        }
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-406-
backend\pipeline\segmenters\yolo_hybrid.py:407:        chunks_dir = os.path.join("output",
"chunks_yolo")
backend\pipeline\segmenters\yolo_hybrid.py:408:        os.makedirs(chunks_dir, exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-409-
backend\pipeline\segmenters\yolo_hybrid.py-410-        # Phase 2: chunk ? per-chunk action
windows ? cross-chunk overlap merge.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py-411-        all_candidate_windows =
self._collect_candidate_windows(
```

11. assistant (2026-06-20T16:56:39.746Z)

Now let me check the detector files to see if they write any artifacts:

12. user (2026-06-20T16:56:40.161Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/audio_features.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/court_gate.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors/fusion_features.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors/native_wasb_runner.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors/person_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/rally_gate.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/stub_runner.py
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors/tracknet_runner.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/wasb_infer.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/wasb_runner.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors/__init__.py
```

13. assistant (2026-06-20T16:56:42.898Z)

Let me read the key detector runner files to find write sites:

14. user (2026-06-20T16:56:43.313Z)

```
1       """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3       This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4       of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5       the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7       The ABC lets us:
8       - Keep the current WSL adapters working unchanged for Windows users.
9       - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11      - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14      Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15      - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16        Run detector, return the Windows-side path to the produced trajectory CSV
17        (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18        The caller (hybrid) is responsible for output dir; impls may stage the video
```

```
        into WSL fs for perf.

    - healthcheck() -> tuple[bool, str]
      Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
      Message is logged (and can be surfaced to reports or UX in future).
      Intended to be cheap (env existence, import, GPU visibility, weights present).

    Existing behavior is 100% preserved. The only additions are the inheritance and
    the explicit common type for future implementers.

    How to add a new runner (for docs / future contributors):
    1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
    2. Implement run_predict and healthcheck (return (bool, str) tuple).
    3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
       deliberately static and model-agnostic). If the new detector needs different
       windowing, it can implement its own or we generalize later.
    4. Lazy-import heavy native deps inside the methods (so importing the segmenter
       registry never pulls torch/tf on a machine that doesn't have them).
    5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
       DetectorRunner instances (they already do the right thing via duck-typing + the
       type hints we add here).
    6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
    7. No change to eval/calibration code paths (they consume the CSV + the static
       windowing helpers directly).

    Optional Linux-native path (the main de-risk goal):
    A LinuxNativeRunner (or ONNXRunner) would:
    - Skip all wsl / _stage / conda activate.
    - Load weights locally (torch or onnx) or call a native binary.
    - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
    - healthcheck can just check "weights file exists and torch importable + CUDA?".
    This removes the WSL requirement for Linux users / future SaaS workers while
    keeping the exact same candidate quality characteristics (or better, once native
    weights are available).

    Error surfacing into reports:
    The healthcheck message is already logged by the hybrids on failure. The reporting
    harvest sees the outcome as "0 candidates" + detector name. When we wire richer
    segmenter Results (or pass detector_health into emit_indexer_report context), the
    (ok, msg) can be recorded explicitly under segmentation.details or a new detector
    stage. The ABC makes that future change local to one place.

    See also:
    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
    """

    import logging
    import shlex
    import subprocess
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional, Tuple

    logger = logging.getLogger(__name__)


    def wsl_tilde_quote(path: str) -> str:
        """shlex.quote that still lets bash expand a leading ``~/``.

        ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
        treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
        directory" ? this broke live WSL staging for the default ``~/clips``
```

```
        stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
        "shlex-quote breaks ~ expansion" item). Quoting only the part after
        ``~/`` keeps expansion AND protects spaces/specials in the remainder.
        """
        if path == "~":
            return path
        if path.startswith("~/"):
            return "~/" + shlex.quote(path[2:])
        return shlex.quote(path)


    class WslCommandMixin:
        """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.

        Lives beside (not on) DetectorRunner: a future Linux-native runner must not
        inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
        and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
        ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
        """

        cfg: Any

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(argv, capture_output=True, text=True,
                                  timeout=(self.cfg.timeout_sec or None))

        # ---- cache hygiene helpers (shared by both WSL runners) ------------------
        def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
            """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.

            Returns None if it can't be determined (never raises). Used as a pre-run
            disk guard so a long extraction doesn't fill the disk mid-flight.
            """
            try:
                res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
'NR==2{{print $4}}'")
            except (subprocess.TimeoutExpired, OSError):
                return None
            try:
                return int((res.stdout or "").strip()) / (1024 ** 3)
            except (ValueError, AttributeError):
                return None

        def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
            """Best-effort recursive delete of WSL paths (post-success cleanup; never
raises)."""
            paths = [p for p in wsl_paths if p]
            if not paths:
                return
            quoted = " ".join(wsl_tilde_quote(p) for p in paths)
            try:
                self._wsl_bash(f"rm -rf {quoted}")
            except (subprocess.TimeoutExpired, OSError) as e:
                logger.warning("WSL cache cleanup failed (non-fatal): %s", e)

        def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
            """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.

            Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
            filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
            fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
            trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
```

```
147             A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
any
148             frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
149             ``cp`` still gets its chance (we never block a genuinely-readable source).
150             """
```

15. user (2026-06-20T16:56:43.572Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45          @classmethod
46          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47              tn = (idx_cfg or {}).get("tracknet", {}) or {}
48              return cls(
49                  distro=tn.get("wsl_distro", "Ubuntu"),
```

```
                repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 1800)),
                keep_frames=bool(tn.get("keep_frames", False)),
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
```

```
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
```

```
176                                 "consider running tools/cleanup_caches.py.",
177                                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179             cmd = (
180                 f"conda activate {self.cfg.conda_env} && "
181                 f"cd {self.cfg.repo_dir}/src && "
182                 f"python predict.py "
183                 f"--video_path {wsl_tilde_quote(wsl_video)} "
184                 f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185                 f"--output_dir {wsl_tilde_quote(wsl_out)} "
186                 f"--queue_length {self.cfg.queue_length}"
187             )
188             logger.info("Running TrackNet inference on %s ...", base)
189             try:
190                 res = self._wsl_bash(cmd)
191             except subprocess.TimeoutExpired:
192                 logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193                 return None
194
195             if res.returncode != 0:
196                 logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197                 return None
198
199             if not os.path.exists(expected_csv_win):
200                 logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
```

16. user (2026-06-20T16:56:43.730Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
```

```
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41          keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
42          min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
43          # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44          # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45          # extraction that dominates a long clip. Output is bit-identical to the PNG path
46          # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47          stream_video: bool = False
48
49          @classmethod
50          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51              w = (idx_cfg or {}).get("wasb", {}) or {}
52              return cls(
53                  distro=w.get("wsl_distro", "Ubuntu"),
54                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56                  conda_env=w.get("conda_env", "wasb"),
57                  weights_path=w.get("weights_path",
58                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59                  sport=w.get("sport", "badminton"),
60                  wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61                  timeout_sec=int(w.get("timeout_sec", 1800)),
62                  keep_frames=bool(w.get("keep_frames", False)),
63                  min_free_gb=float(w.get("min_free_gb", 0.0)),
64                  stream_video=bool(w.get("stream_video", False)),
65              )
66
67
68      class WasbRunner(WslCommandMixin, DetectorRunner):
69          def __init__(self, cfg: WasbConfig):
70              self.cfg = cfg
71
72          def healthcheck(self) -> Tuple[bool, str]:
73              """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74              cmd = (f"conda activate {self.cfg.conda_env} && "
75                     f"python -c \"import torch; print('torch', torch.__version__, "
76                     f"'cuda', torch.cuda.is_available())\"")
77              try:
78                  res = self._wsl_bash(cmd)
79              except FileNotFoundError:
80                  return False, "`wsl` not found ? Windows + WSL2 required."
81              except subprocess.TimeoutExpired:
82                  return False, "WSL healthcheck timed out."
83              out = (res.stdout or "").strip()
84              if res.returncode != 0:
85                  return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86              return True, out
87
88          def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89              src_mnt = to_wsl_mnt_path(video_win_path)
90              reason = self.wsl_source_unreadable_reason(src_mnt)
91              if reason:
```

```
92                  logger.error("Cannot stage video into WSL: %s", reason)
93                  return None
94              dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95              cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96              try:
97                  res = self._wsl_bash(cmd)
98              except subprocess.TimeoutExpired:
99                  logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)
111             return res.returncode == 0 and "OK" in (res.stdout or "")
112
113         def run_predict(self, video_win_path: str, output_win_dir: str,
114                         video_id: Optional[str] = None) -> Optional[str]:
115             """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117             All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118             two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119             can never reuse each other's staged frames, resumable cache, or CSV.
120             """
121             # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122             # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123             # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
124             # the CSV landed inside WSL while Windows polled the repo-relative path.
125             output_win_dir = os.path.abspath(output_win_dir)
126             os.makedirs(output_win_dir, exist_ok=True)
127             # Normalize for cross-platform basename (tests use Windows paths on Linux).
128             video_win_path = str(video_win_path).replace("\\", "/")
129             base = os.path.basename(video_win_path)
130             stem, ext = os.path.splitext(base)
131             # Unique, content-derived key: same filename + different bytes -> different key.
132             key = f"{stem}__{video_id[:12]}" if video_id else stem
133             wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134             expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136             if not self._stage_infer_script():
137                 logger.error("Could not stage wasb_infer.py into WSL.")
138                 return None
139             staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140             if staged_video is None:
141                 return None
142             frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144             # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145             # WSL stage filesystem is already low so the user can clean up before it fills.
146             # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147             if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148                 free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149                 if free is not None and free < self.cfg.min_free_gb:
150                     logger.warning(
151                         "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
```

```
152                         "may fill the disk ? consider running tools/cleanup_caches.py.",
153                         free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155             # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156             # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157             if self.cfg.stream_video:
158                 frame_args = "--stream-video "
159             else:
160                 frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161             cmd = (
162                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163                 f"python wasb_infer.py "
164                 f"--video {wsl_tilde_quote(staged_video)} "
165                 f"{frame_args}"
166                 f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167                 f"--sport {shlex.quote(self.cfg.sport)} "
168                 f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
169             )
170             logger.info("Running WASB inference on %s ...", base)
171             try:
172                 res = self._wsl_bash(cmd)
173             except subprocess.TimeoutExpired:
174                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175                 return None
176             if res.returncode != 0:
177                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178                 return None
179             if not os.path.exists(expected_csv_win):
180                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181                 return None
182             logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189             if not self.cfg.keep_frames:
190                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191                 logger.info("Cache hygiene: removing staged video%s for %s "
192                             "(set indexing.wasb.keep_frames=true to retain).",
193                             "" if self.cfg.stream_video else " + frame cache", key)
194                 self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200
```

17. assistant (2026-06-20T16:56:46.062Z)

Let me also check the person_detector.py to see if it writes artifacts:

18. user (2026-06-20T16:56:46.518Z)

```
1       """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
```

```
3       Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4       the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5       `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6       next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7       later the learned classifier reads its persisted features. Keeping detection here (not
in
8       the segmenter) is what makes "add a cue = register a producer" true.
9
10      Behaviour is unchanged from the in-lined version ? only the home moved:
11        - `lazy_load_model`               ? load a permissively-licensed torchvision detector
12        - `make_tracker`                  ? a fresh supervision ByteTrack per chunk
13        - `detect_and_track`              ? one frame ? confident persons with persistent IDs
14        - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15          with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16          that feed the candidate `stats` JSON and the learned fusion features.
17
18      The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19      detector weights load only on first detection (`lazy_load_model`) ? so importing the
20      segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21      are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22      machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23      (ADR-005; ultralytics' AGPL YOLO was dropped).
24      """
25      import logging
26      from typing import Any, Callable, Dict, List, Optional, Tuple
27
28      import cv2
29      import torch
30
31      from backend.utils.geometry import get_velocity_normalised
32
33      logger = logging.getLogger(__name__)
34
35      # torchvision detection models are trained on COCO where the "person" category is
36      # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37      # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38      _PERSON_CLASS_ID = 1
39
40      # Permissively-licensed torchvision detectors selectable via config
41      # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42      _DETECTOR_FACTORIES = {
43          "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44          "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45          "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46      }
47      _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50      class PersonDetector:
51          """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53          Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54          ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55          min_action_window_duration}``); the model loads lazily on first use.
56          """
57
58          def __init__(self, config: Dict[str, Any]):
59              self.config = config
60              # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61              # `self.model([tensor])` inference call type-checks without a quarantine.
62              self.model: Any = None
63              self.device: Optional[str] = None
```

```
64
65          def lazy_load_model(self) -> None:
66              if self.model is not None:
67                  return
68
69              # Imported lazily so the module imports without torchvision present and so
70              # the test suite (which pre-sets self.model) never pulls in model weights.
71              from torchvision.models import detection as tv_detection
72
73              idx_cfg = self.config.get("indexing", {})
74              use_gpu = idx_cfg.get("use_gpu", True)
75              self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76              if use_gpu and not torch.cuda.is_available():
77                  logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79              model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80              if model_name not in _DETECTOR_FACTORIES:
81                  logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82                  model_name = _DEFAULT_DETECTOR
83
84              builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85              logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86              # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87              self.model = builder(weights="DEFAULT")
88              self.model.eval()
89              self.model.to(self.device)
90
91          def make_tracker(self, fps: float) -> Any:
92              """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94              Imported lazily so tests can mock this method without supervision installed.
95              """
96              import supervision as sv
97              frame_rate = max(1, int(round(fps)))
98              return sv.ByteTrack(frame_rate=frame_rate)
99
100         def detect_and_track(self, frame: Any, tracker: Any,
101                              score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102             """Detect persons in a frame and assign persistent track IDs.
103
104             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105             This replaces the old ultralytics ``model.track(...)`` call, which bundled
106             detection AND association: torchvision does the detection, supervision's
107             ByteTrack does the association.
108             """
109             import supervision as sv
110
111             # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113             tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114             with torch.no_grad():
115                 outputs = self.model([tensor])
116
117             out = outputs[0]
118             boxes = out["boxes"].detach().cpu().numpy()
119             labels = out["labels"].detach().cpu().numpy()
120             scores = out["scores"].detach().cpu().numpy()
121
122             # Keep only confident person detections.
123             mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124             if not mask.any():
```

```
125                 return []
126
127             # ByteTrack requires confidence to be populated (it filters on it internally).
128             dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129             tracked = tracker.update_with_detections(dets)
130
131             result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132             for i in range(len(tracked)):
133                 tid = tracked.tracker_id[i]
134                 if tid is None:
135                     continue
136                 x1, y1, x2, y2 = tracked.xyxy[i]
137                 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138             return result
139
140         def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                               velocity_thresh: float, merge_gap: float,
142                                               frame_skip: int = 3,
143                                               chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
144             """
145             Runs person detection + tracking on a video chunk and extracts high-motion
146             action windows. Returns a list of dicts in the full video's timeframe, each
with:
147                 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148                 track_id_count, chunk_index.
149
150             Args:
151                 frame_skip: Process every Nth frame (1 = every frame). Higher values
152                             dramatically reduce inference cost with minimal quality loss.
153                 chunk_index: Index of the source chunk (recorded on each window for
traceability).
154             """
155             cap = cv2.VideoCapture(chunk_path)
156             if not cap.isOpened():
157                 logger.error(f"Could not open {chunk_path}")
158                 return []
159
160             fps = cap.get(cv2.CAP_PROP_FPS)
161             if fps <= 0:
162                 fps = 30.0
163
164             frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165             if frame_width <= 0:
166                 frame_width = 1280.0  # Sensible fallback
167
168             # Effective sampling rate after frame-skipping
169             effective_fps = fps / max(frame_skip, 1)
170
171             # A fresh tracker per chunk ? track IDs only need to be consistent within a
172             # chunk (windows are computed per chunk, then merged across chunks by time).
173             tracker = self.make_tracker(effective_fps)
174             score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176             # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177             # so windows can be enriched with motion stats for logging + fine-tuning.
178             active_frames = []
179             frame_idx = 0
180             track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182             while True:
183                 ret, frame = cap.read()
184                 if not ret:
```

```
                    break

                # Skip frames for performance ? at 30fps, every 3rd frame still
                # gives ~10 samples/second, plenty for human-scale velocity.
                if frame_idx % max(frame_skip, 1) != 0:
                    frame_idx += 1
                    continue

                # Detect persons + assign persistent track IDs.
                detections = self.detect_and_track(frame, tracker, score_thresh)

                frame_peak_velocity = 0.0
                frame_active_ids = set()
                current_ids = set()

                for track_id, bbox in detections:
                    current_ids.add(track_id)

                    if track_id in track_history:
                        prev_bbox = track_history[track_id]
                        velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
                        if velocity > velocity_thresh:
                            frame_peak_velocity = max(frame_peak_velocity, velocity)
                            frame_active_ids.add(track_id)

                    track_history[track_id] = bbox

                # Prune stale IDs that are no longer visible ? prevents false
                # velocity spikes when the tracker reassigns an old ID.
                track_history = {k: v for k, v in track_history.items() if k in current_ids}

                if frame_active_ids:
                    active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))

                frame_idx += 1

            cap.release()

            # Group active frames into continuous windows
            if not active_frames:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
                velocities = [v for _, v, _ in group]
                track_ids = set().union(*[ids for _, _, ids in group])
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(velocities),
                    "mean_velocity": sum(velocities) / len(velocities),
                    "track_id_count": len(track_ids),
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active_frames[0]]
            for af in active_frames[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
```

```
249                     group = [af]
250             windows.append(_finalize(group))
```

19. assistant (2026-06-20T16:56:48.657Z)

Now let me search for all write operations more comprehensively:

20. user (2026-06-20T16:56:49.770Z)

```
<persisted-output>
Output too large (20.3KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-
Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\tool-
results\toolu_01Fz5oep3a1FmdPZj9qLyuge.txt

Preview (first 2KB):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-23-
# normalized to a dict (or None) so this module stays config-package-agnostic.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-24-
self.watermark = self._normalize_watermark(watermark)
backend\pipeline\stitcher\compiler.py:25:        os.makedirs(output_dir, exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-26-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-27-
@staticmethod
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-163-
# textfile= sidesteps the notorious drawtext text= escaping for arbitrary strings.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-164-
tf_path = os.path.join(tmp_dir, "watermark.txt")
backend\pipeline\stitcher\compiler.py:165:            with open(tf_path, "w", encoding="utf-8")
as fh:
backend\pipeline\stitcher\compiler.py:166:                fh.write(text)
backend\pipeline\stitcher\compiler.py-167-            # Font precedence: explicit font_path ?
bundled Apache-2.0 default (renders
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-168-
# without fontconfig) ? fontconfig family (last-resort, environment-dependent).
--
backend\pipeline\stitcher\compiler.py-376-            # encoding pinned: a non-ASCII clip path
would crash on Windows (cp1252 default);
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py-377-
# ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
backend\pipeline\stitcher\compiler.py:378:            with open(list_file_path, "w",
encoding="utf-8") as f:
backend\pipeline\stitcher\compiler.py-379-                for clip in temp_clips:
...
</persisted-output>
```

21. assistant (2026-06-20T16:56:53.004Z)

Let me also check for any CSV writing in the detectors:

22. user (2026-06-20T16:56:53.552Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/base.py:  (e.g.
".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/native_wasb_runner.py:It emits the identical
``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/native_wasb_runner.py:        runner does, and the CSV is
named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte the
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/native_wasb_runner.py:        out_csv =
os.path.join(output_dir, f"{key}_wasb.csv")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/stub_runner.py:
```

```
out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/stub_runner.py:
w = _csv.writer(f)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/stub_runner.py:
w = _csv.writer(f)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/tracknet_runner.py:        therefore predict.py's
``<stem>_predict.csv`` output ? so two videos that share
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/tracknet_runner.py:        # predict.py only accepts
.avi/.mp4 and names the CSV "<stem>_predict.csv".
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/tracknet_runner.py:        # under the namespaced name
propagates into "<key>_predict.csv".
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/tracknet_runner.py:        expected_csv_win =
os.path.join(output_win_dir, f"{key}_predict.csv")
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/tracknet_runner.py:            # No staging: predict.py reads
/mnt directly and writes "<stem>_predict.csv".
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline/detectors/tracknet_runner.py:            expected_csv_win =
os.path.join(output_win_dir, f"{stem}_predict.csv")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/wasb_infer.py:
--out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-every 1000]
[--fresh]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/wasb_infer.py:
w = csv.writer(f)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/wasb_runner.py:
expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline/detectors/wasb_runner.py:
f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
```

23. assistant (2026-06-20T16:56:55.431Z)

Now let me look at the stub_runner and native_wasb_runner more carefully:

24. user (2026-06-20T16:56:55.656Z)

```
1       """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3       A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4       CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
5       (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6       container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7       (``use_gpu=False``); this closes the last GPU/WSL-only gap so the *entire* pipeline is
8       runnable ? and regression-gateable ? without a GPU.
9
10      Two modes:
11        * **replay** ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12          pipeline with a committed/golden trajectory; deterministic + cross-OS).
13        * **synth** (default) ? generate a deterministic synthetic trajectory of
14          ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15          gaps, so windowing yields a stable, known number of candidate windows.
16
17      This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18      "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19      (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20      ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21      WSL runners stay the default, so production behaviour is unchanged.
22      """
23      import os
24      import csv as _csv
25      import shutil
26      import logging
```

```
    from dataclasses import dataclass
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.detectors.base import DetectorRunner
    # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    logger = logging.getLogger(__name__)


    @dataclass
    class StubConfig:
        """Settings for the CPU stub runner (built from ``config.json`` ->
``indexing.stub``)."""
        fps: float = 30.0
        frame_width: float = 1280.0
        num_rallies: int = 2
        rally_seconds: float = 4.0
        gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
windows don't merge)
        lead_seconds: float = 2.0         # dead lead-in
        step_px: float = 12.0             # per-frame shuttle displacement during a rally
(>> velocity_thresh)
        replay_csv: Optional[str] = None   # if set + exists, emit this CSV verbatim instead
of synthesising

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
            s = (idx_cfg or {}).get("stub", {}) or {}
            return cls(
                fps=float(s.get("fps", 30.0)),
                frame_width=float(s.get("frame_width", 1280.0)),
                num_rallies=int(s.get("num_rallies", 2)),
                rally_seconds=float(s.get("rally_seconds", 4.0)),
                gap_seconds=float(s.get("gap_seconds", 3.0)),
                lead_seconds=float(s.get("lead_seconds", 2.0)),
                step_px=float(s.get("step_px", 12.0)),
                replay_csv=s.get("replay_csv"),
            )


    class StubDetectorRunner(DetectorRunner):
        """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""

        def __init__(self, cfg: Optional[StubConfig] = None):
            self.cfg = cfg or StubConfig()

        def healthcheck(self) -> Tuple[bool, str]:
            return True, "stub CPU detector (no GPU/WSL)"

        def _rows(self) -> List[Tuple[int, int, float, float]]:
            """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
bursts
            separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
?
            well above the default 0.02 velocity threshold ? so every rally frame is
"active"
            and each burst becomes exactly one candidate window."""
            cfg = self.cfg
            rows: List[Tuple[int, int, float, float]] = []
            frame = 0
            y = cfg.frame_width / 4.0
            base_x = cfg.frame_width / 2.0
            for _ in range(int(cfg.lead_seconds * cfg.fps)):
                rows.append((frame, 0, 0.0, 0.0))
```

```
85                  frame += 1
86              for r in range(cfg.num_rallies):
87                  for i in range(int(cfg.rally_seconds * cfg.fps)):
88                      x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
89                      rows.append((frame, 1, x, y))
90                      frame += 1
91                  if r != cfg.num_rallies - 1:
92                      for _ in range(int(cfg.gap_seconds * cfg.fps)):
93                          rows.append((frame, 0, 0.0, 0.0))
94                          frame += 1
95              return rows
96
97          def run_predict(self, video_win_path: str, output_win_dir: str,
98                          video_id: Optional[str] = None) -> Optional[str]:
99              """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
100             os.makedirs(output_win_dir, exist_ok=True)
101             stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
"/")))[0]
102             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
103             out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
104
105             if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
106                 shutil.copyfile(self.cfg.replay_csv, out_csv)
107                 logger.info("Stub runner: replayed trajectory %s -> %s",
self.cfg.replay_csv, out_csv)
108                 return out_csv
109
110             rows = self._rows()
111             with open(out_csv, "w", newline="") as f:
112                 w = _csv.writer(f)
113                 w.writerow(["Frame", "Visibility", "X", "Y"])
114                 for frame, vis, x, y in rows:
115                     w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
116             logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
out_csv)
117             return out_csv
118
119         # Model-agnostic CSV parsing + windowing ? identical to the real runners.
120         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
121         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
122
123
124     def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
125                                      frame_width: float = 1280.0, tail_pad: float = 2.0,
126                                      step_px: float = 12.0) -> str:
127         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
128         in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
129         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
130
131         This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
132         no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
133         learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
134         """
135         rallies: List[Tuple[float, float]] = []
136         with open(labels_csv, newline="", encoding="utf-8") as f:
137             for row in _csv.DictReader(f):
138                 try:
139                     rallies.append((float(row["start_time"]), float(row["end_time"])))
```

```
                except (KeyError, ValueError):
                    continue
        duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
        nframes = max(1, int(duration * fps))
        base_x = frame_width / 2.0
        y = frame_width / 4.0
        os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
        with open(out_csv, "w", newline="", encoding="utf-8") as f:
            w = _csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            for i in range(nframes):
                t = i / fps
                if any(s <= t < e for s, e in rallies):
                    x = base_x + (step_px if i % 2 else -step_px)
                    w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
                else:
                    w.writerow([i, 0, "0.000", "0.000"])
        return out_csv

```

25. user (2026-06-20T16:56:55.949Z)

```
      """Linux-native WASB shuttle-trajectory runner (no WSL).

      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
      Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
      see the acceptance gate below) ? but WITHOUT the WSL transport:

        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
          into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
        * weights / repo / python / scratch come from the **environment** first
          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.

      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.

      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
```

```
33      not the numbers.
34      """
35      import os
36      import shutil
37      import logging
38      import subprocess
39      from dataclasses import dataclass
40      from typing import Any, Dict, List, Optional, Tuple
41
42      from backend.pipeline.detectors.base import DetectorRunner
43      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
44      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
45
46      logger = logging.getLogger(__name__)
47
48      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
49      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
50      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
51
52
53      @dataclass
54      class NativeWasbConfig:
55          """Resolved settings for a Linux-native WASB run.
56
57          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
58          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
59          defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
60          """
61          repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
62          python_bin: str = "python"             # the `wasb` conda env python (invoked by
path, no activate)
63          weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
64          sport: str = "badminton"
65          device: str = "cuda"                   # cuda | mps | cpu
66          scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next
to the output dir
67          timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
timeout)
68          keep_frames: bool = False              # retain the decoded frame cache after
success (debug only)
69          # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
70          # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
71          # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
72          stream_video: bool = True
73
74          @classmethod
75          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
76              w = (idx_cfg or {}).get("wasb", {}) or {}
77              env = os.environ.get
78              # stream_video is tri-state in config: None/absent => native default (stream);
an
79              # explicit True/False still wins (e.g. False for a container cv2 can't seek).
80              sv = w.get("stream_video")
81              return cls(
82                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
83                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
84                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
85                  sport=w.get("sport", "badminton"),
```

```
86                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
87                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
88                  timeout_sec=int(w.get("timeout_sec", 1800)),
89                  keep_frames=bool(w.get("keep_frames", False)),
90                  stream_video=(True if sv is None else bool(sv)),
91              )
92
93
94      class NativeWasbRunner(DetectorRunner):
95          """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
96
97          def __init__(self, cfg: NativeWasbConfig):
98              self.cfg = cfg
99
100         # --- low-level native exec (the single place tests patch)
--------------------------
101         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
102             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
103             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
104                                   timeout=(self.cfg.timeout_sec or None))
105
106         def _repo_src(self) -> str:
107             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
108
109         def _copy_infer_script(self) -> bool:
110             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
111             repo_src = self._repo_src()
112             try:
113                 os.makedirs(repo_src, exist_ok=True)
114                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
115                 return True
116             except OSError as e:
117                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
118                 return False
119
120         def _rm_rf(self, path: Optional[str]) -> None:
121             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
122             if path:
123                 shutil.rmtree(path, ignore_errors=True)
124
125         def healthcheck(self) -> Tuple[bool, str]:
126             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
127             if not self.cfg.weights_path:
128                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
129             weights = os.path.expanduser(self.cfg.weights_path)
130             if not os.path.exists(weights):
131                 return False, f"WASB weights not found at {weights}"
132             repo_src = self._repo_src()
133             if not os.path.isdir(repo_src):
134                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
135                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
136             code = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
137             try:
138                 res = self._run([self.cfg.python_bin, "-c", code])
139             except FileNotFoundError:
140                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
```

```
WASB_PYTHON)."
            except subprocess.TimeoutExpired:
                return False, "native torch healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
            if self.cfg.device == "cuda" and "cuda True" not in out:
                return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
            return True, out

        def run_predict(self, video_win_path: str, output_win_dir: str,
```

26. assistant (2026-06-20T16:56:58.612Z)

Let me read the rest of native_wasb_runner.py and check for more write locations:

27. user (2026-06-20T16:56:59.458Z)

```
        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.

            Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
            runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
            same downstream contract, so the parity comparison is apples-to-apples.
            """
            output_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_dir, exist_ok=True)
            # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
            norm = str(video_win_path).replace("\\", "/")
            stem, _ext = os.path.splitext(os.path.basename(norm))
            # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
            key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
            out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

            if not self._copy_infer_script():
                logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
                return None

            weights = os.path.expanduser(self.cfg.weights_path)
            argv = [self.cfg.python_bin, "wasb_infer.py",
                    "--video", os.path.abspath(str(video_win_path))]
            frames_dir: Optional[str] = None
            if self.cfg.stream_video:
                # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
                argv.append("--stream-video")
            else:
                scratch = self.cfg.scratch_dir or output_dir
                frames_dir = os.path.join(scratch, f"{key}_frames")
                argv += ["--frames_out_dir", frames_dir]
            argv += ["--weights", weights,
                     "--sport", self.cfg.sport,
                     "--device", self.cfg.device,
                     "--out", out_csv]

            logger.info("Running native WASB inference (device=%s) on %s ...",
                        self.cfg.device, os.path.basename(norm))
            try:
                res = self._run(argv, cwd=self._repo_src())
```

```
191             except FileNotFoundError:
192                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
193                 return None
194             except subprocess.TimeoutExpired:
195                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
196                 return None
197             if res.returncode != 0:
198                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
199                 return None
200             if not os.path.exists(out_csv):
201                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
202                 return None
203             logger.info("WASB trajectory CSV: %s", out_csv)
204
205             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
206             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
207             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
208             if not self.cfg.keep_frames and frames_dir:
209                 logger.info("Cache hygiene: removing frame cache for %s "
210                             "(set indexing.wasb.keep_frames=true to retain).", key)
211                 self._rm_rf(frames_dir)
212             return out_csv
213
214         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
215         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
216         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
217
```

28. user (2026-06-20T16:56:59.874Z)

```
100:    with open(tmp, "w") as f:
131:        with open(mpath) as f:
178:        with open(det_jsonl, "r") as f:
208:    with open(tmp, "w", newline="") as f:
308:    The disk path is ``frame_bgr -> cv2.imwrite(PNG) -> Image.open(PNG).convert('RGB')``;
340:    ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
388:      to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
494:        det_f = open(det_jsonl, "a")
```

29. assistant (2026-06-20T16:57:02.488Z)

Now let me search more comprehensively for all file write operations in the pipeline:

30. user (2026-06-20T16:57:03.744Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters/gemini.py:119:
temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
```

31. assistant (2026-06-20T16:57:06.216Z)

Now let me create a comprehensive summary by examining the exact lines we found. Let me verify the key artifact locations one more time by examining the code line by line:

32. user (2026-06-20T16:57:06.566Z)

```
15
16      def _chunk_workdir(video_id: str) -> str:
17          """Per-video chunk staging dir under output/chunks/<video_id>.
18
19          Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20          otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21          in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22          """
23          return os.path.join("output", "chunks", video_id)
24
25
26      def _parse_time(val) -> float:
27          """Converts a timestamp value to total float seconds.
28          Handles float/int pass-through, plain numeric strings, and
29          colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
30          when unexpected formats are encountered."""
31          if isinstance(val, (int, float)):
32              return float(val)
33          if isinstance(val, str):
34              val = val.strip()
35              if ":" in val:
36                  # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
37                  # Convert it, but warn so the issue is visible in logs.
38                  parts = val.split(":")
39                  parts.reverse()
40                  total = 0.0
41                  for i, p in enumerate(parts):
42                      try:
43                          total += float(p) * (60 ** i)
44                      except ValueError:
45                          pass
46                  logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
47                  return total
48              try:
49                  return float(val)
50              except ValueError:
51                  logger.warning(f"Could not parse timestamp string '{val}' as a number.
Defaulting to 0.0.")
52                  return 0.0
53          logger.warning(f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0.")
54          return 0.0
55
56
57      class GeminiPlaySegmenter(BasePlaySegmenter):
58          @property
59          def name(self) -> str:
60              return "gemini"
61
62          @property
63          def description(self) -> str:
64              return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
```

33. assistant (2026-06-20T16:57:09.002Z)

Now let me compile all the findings. Let me verify the exact line numbers for the two hardcoded paths mentioned:

34. user (2026-06-20T16:57:09.326Z)

```
313:            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
```

35. user (2026-06-20T16:57:11.958Z)

```
237:        out_dir = os.path.join("output", self.family)
313:            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
```

36. assistant (2026-06-20T16:57:14.036Z)

Perfect! Now let me verify the yolo_hybrid and gemini paths:

37. user (2026-06-20T16:57:14.510Z)

```
101:        frame_skip: int, chunks_dir: str,
132:                chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
170:                chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
268:        gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
407:        chunks_dir = os.path.join("output", "chunks_yolo")
408:        os.makedirs(chunks_dir, exist_ok=True)
414:            frame_skip, chunks_dir,
423:            os.rmdir(chunks_dir)
```

38. user (2026-06-20T16:57:17.190Z)

```
16:def _chunk_workdir(video_id: str) -> str:
23:    return os.path.join("output", "chunks", video_id)
119:            temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
536:        chunks_dir = _chunk_workdir(video_id)
```

39. assistant (2026-06-20T16:57:19.827Z)

Now I have all the information. Let me compile the comprehensive mapping:

40. user (2026-06-20T16:57:42.032Z)

Structured output provided successfully

41. assistant (2026-06-20T16:57:49.029Z)

Summary

I've completed a comprehensive mapping of every per-video artifact write site in the LOCAL pipeline/segmenter path. Here are the key findings:

****Hardcoded Paths (2 literal "output" patterns)****
1. **Line 237 (trajectory_hybrid.py)**: `out_dir = os.path.join("output", self.family)` ? stores trajectory CSVs keyed by detector family
2. **Line 313 (trajectory_hybrid.py)**: `handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")` ? stores padded clips for AI validation

****Additional Hardcoded Paths****
- **Line 407 (yolo_hybrid.py)**: `chunks_dir = os.path.join("output", "chunks_yolo")` ? person-detection chunks
- **Line 23 (gemini.py)**: `return os.path.join("output", "chunks", video_id)` ? per-video gemini chunk subdir
- **Line 119 (gemini.py)**: `temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")` ? debug trimming feature

****Artifact Categories Mapped****
1. **Trajectory CSVs** (Frame,Visibility,X,Y) ? written by detectors via `run_predict()`, keyed by `video_id[:12]` in filename
2. **Candidate segments** ? persisted to SQLite DB, keyed by `video_id`
3. **Handoff clips** ? temporary MP4s for AI validation, keyed by `video_id + window_idx`, deleted post-processing
4. **Frame cache** ? WSL PNG extraction (`~/clips/<video_id>_frames/`), deleted unless

`keep_frames=true`
5. **Staged videos** ? WSL copies for performance (`~/clips/<video_id>.<ext>`), deleted unless `keep_frames=true`
6. **Play segments** ? final validated rallies in DB, linked back to candidate sources

All paths use relative `"output"` as the root directory and are parameterized by `video_id` (content MD5) or `family`/`provider_name` to avoid collisions and support resumable cache behavior.